PYTHON CASE STUDIES — EASY ONE-PAGE NOTES

The common idea: take input → store data → apply rules → process items → show results.

1. Student Marks Matrix

Store marks in a **nested list** (each row is one student). Use loops to calculate each student's total/average and each subject's average. Compare averages to find the top performer and subjects with low averages.

2. Employee Shift Conflict

Store each shift as a **tuple**, such as `(employee, start, end)`. Compare shifts. If one shift starts before another ends and ends after it starts, they overlap. Report the conflicting shifts.

3. Product Catalogue Audit

Each product is a **tuple** with details such as name, price and quantity. **Unpack** the fields, check that values are valid, calculate stock value (**price × quantity**), and list products that need replenishment.

4. Smart Canteen Order

Keep item names, prices and stock in **lists/variables**. For each order, check requested quantity against stock. If available, calculate the bill and reduce stock; otherwise reject the unavailable quantity. Finally, display the order summary.

Common Algorithm Structure

1. INPUT Read required values.	**2. INITIALIZE** Create variables, lists or tuples.	**3. PROCESS** Use loops, operators and conditions.
4. CALCULATE Find totals, averages or checks.	**5. VALIDATE** Handle invalid/unavailable data.	**6. OUTPUT** Display results and stop.

Simple Pseudocode Pattern

```
START
INPUT required data
INITIALIZE variables / lists / tuples
FOR each item:
    CHECK conditions and perform calculations
    UPDATE totals or results
END FOR
DISPLAY final result
STOP
```

Remember: lists store multiple values; nested lists store rows of data; tuples group related fields; loops repeat work; conditions make decisions.